STL-FEM-Solver

Moritz Jakob, Til Gramlich

Ruprecht-Karls-Universität Heidelberg
Heidelberg, Germany
moritz.jakob02@stud.uni-heidelberg.de
til.gramlich@stud.uni-heidelberg.de

***Abstract*—This project presents an interactive finite element analysis application designed to perform structural simulations directly on STL geometries. The main goal was to create a tool that simplifies the setup of FEM problems and helps users to interpret the resulting structural behaviour. The program includes a graphical user interface with a 3D viewer that allows users to inspect the mesh, apply loads, constraints and adjust simulation parameters. Visualization options are integrated to display the mesh, boundary conditions, the computed displacement and stress fields. The solver is based on linear elasticity and supports both point and area loads. To keep computations efficient, the program automatically switches between a direct and an iterative solver depending on the mesh size. Validation using analytical benchmark problems showed good agreement with the reference solution, with errors below 1% on the finest mesh. Overall, the application demonstrates that finite element simulations on STL-based geometries can be carried out in a practical and accessible way.**

INTRODUCTION

Designing reliable products, such as machine parts, support structures, civil structures or medical implants, requires understanding how they behave under real forces. If this behavior is predicted incorrectly, the result can be structural failures or inefficient designs that lead to unnecessary costs. In the past, engineers mainly relied on repeatedly building and testing physical prototypes, which was both time-consuming and expensive. To reduce these limitations, modern workflows increasingly rely on computer simulation. One of the most widely used techniques is the Finite Element Method (FEM), which divides a complex structure into many small elements and computes how each element responds to forces and constraints based on its material behavior. By combining these local solutions, FEM can predict how a real object will deform, where stresses will appear and therefore how safe a design is.

In many fields today, 3D models are defined in the STL format. They are easy to generate and especially common in additive manufacturing and rapid prototyping. However, they only describe the surface geometry of an object and are not designed for volumetric simulation. As a result, most FEM tools require additional preprocessing steps or format conversions before a simulation can begin. These requirements make simple experiments unnecessarily difficult.

The aim of this project was to reduce these barriers by creating a user-friendly application that can run basic structural FEM simulations directly on STL files. The application provides a graphical user interface with an interactive 3D view, allowing users to select parts of the mesh and assign them loads or boundary conditions through direct interaction. All steps, from importing the model and refining the mesh to setting parameters and inspecting results, are organized within an intuitive layout that supports a smooth workflow. The solver is based on linear elasticity and supports both point and area loads, which can be assigned by selecting the corresponding surface regions. An adaptive strategy switches between a direct or an iterative solver depending on the mesh size to keep computations efficient.

After the simulation, users can visualize displacement fields, the deformed shape of the structure and stress and strain components. These tools make the results easier to interpret and provide insight into the mechanical behavior of the analyzed geometry. To evaluate the implementation, a benchmark problem with known analytical solutions was tested. The solver showed good agreement with the reference results, reaching an error below 1% on the finest mesh.

The remainder of this report first reviews related work and situates the project among existing FEM tools. It then describes the architecture and implementation of the application, followed by the validation experiments. The report ends with suggestions for possible future improvements.

RELATED WORK

The project is situated at the intersection of three active areas of research. These include mesh generation from STL geometries, the development of open-source finite element frameworks and the creation of interactive tools for simulation setup and visualization. This section places the developed STL-based finite element solver within that broader context.

STL-based Meshing and Geometry Preparation

A key challenge is the transformation of STL surface meshes into volumetric meshes that are suitable for finite

element analysis, especially when the input geometry is noisy or of low quality. Béchet et al. [1] show that STL data can contain defects such as holes, gaps, overlaps and topological inconsistencies that complicate reliable mesh generation. Their approach focuses on recovering the surface topology and generating a conforming remeshed surface from STL data so that it can serve as a valid basis for finite element meshing.

This project encounters similar limitations. As in the study by Béchet et al. [1], failures in the meshing process mostly arise from geometric inconsistencies in the input files. While their work introduces a more systematic remeshing approach, the application developed here instead relies on a simplified, automated pipeline in which STL files are imported, optionally refined and then passed directly to Gmsh. This makes robust geometry handling an important area for future improvement.

Müller et al. [2] examine a related topic from a more graphics-oriented perspective. They propose techniques for dynamically generating volumetric meshes from surface meshes to support physically based simulation of deformable and fracturing objects. Although their focus is on real-time animation rather than engineering analysis, the underlying requirement of converting surface meshes into volumetric representations is directly relevant to this project.

Overall, these studies highlight two key aspects relevant to this project. The first is the reliance on tetrahedral meshing when working with surface-based STL input. The second is the importance of mesh quality and geometry handling for successful finite element simulation.

Finite Element Method and FEniCS

The numerical core of the project is based on the Finite Element Method, a widely used technique for solving partial differential equations in engineering and applied sciences. A foundational reference is the formulation presented by Zienkiewicz and Taylor [3], who provide a comprehensive mathematical treatment of the method for structural and continuum mechanics. Their work formalizes the conversion of governing equations into a weak form and the construction of discrete approximations over tetrahedral or hexahedral meshes. This formulation underlies the variational expressions used in the project, where displacement fields, strains and stresses are computed using a small-strain linear elastic model.

Building on this theoretical foundation, the FEniCS Project offers an automated framework for implementing the Finite Element Method for a wide range of partial differential equations. Alnæs et al. [4] describe FEniCS as a collection of interoperable components capable of translating high-level variational formulations into optimized low-level finite element code. The Unified Form Language allows users to express weak forms in a syntax close to mathematical notation, while the FEniCS compiler manages symbolic differentiation, element-level integration, matrix assembly and interfacing with efficient linear algebra backends.

This project makes direct use of these capabilities. The linear elasticity problem is formulated in the Unified Form Language and FEniCS generates the stiffness matrices, applies boundary conditions and solves the resulting linear system using either direct or iterative methods. As a result, the implementation remains compact while maintaining numerical reliability. Compared to traditional script-based workflows, the system developed here integrates the FEniCS backend into a graphical interface operating directly on STL geometries. This lowers the barrier for users without prior experience in partial differential equation programming.

Interactive Meshing and Simulation Tools

There is also prior work on interactive tools that combine mesh generation, model setup, simulation and post-processing. FEATool Multiphysics [5] is an example of a graphical environment that provides a GUI-based workflow with integrated support for FEniCS as one of its solver backends. FEATool provides geometry definition and multiphysics setup through a graphical interface with native MATLAB integration and it can export or convert models to FEniCS Python simulation scripts. In comparison, the system developed here has a narrower focus. It targets structural simulations carried out directly on STL geometries and exposes the underlying mesh more explicitly by enabling interactive selection of vertices and faces for loads and boundary conditions. While FEATool aims to deliver a broad multiphysics modeling environment, this project concentrates on a workflow for STL-based linear elasticity with particular emphasis on control over mesh regions and immediate visual feedback.

General-purpose open-source finite element software such as Elmer further demonstrate how user-oriented interfaces can be combined with advanced numerical solvers to support efficient and flexible simulation workflows [6].

Taken together, these related works show that the main components relevant to this project, including mesh generation, finite element solution workflows and interactive simulation environments, have been explored in earlier literature. This project combines these ideas into an accessible end-to-end tool for finite element simulations directly on STL geometries while relying on established open-source technologies for meshing and numerical analysis.

METHODS AND IMPLEMENTATION

This section describes the methodological foundations and software architecture of the developed system. It outlines how components such as geometry handling, meshing, load assignment, numerical solution and visualization are combined into a workflow for finite element analysis on STL geometries.

System Architecture and Workflow

The STL FEM Solver was built as an end-to-end tool that takes the user from importing an STL file to viewing the simulation results. The workflow starts when the user loads an STL surface mesh through the menu bar. This menu also includes options for opening previously generated XDMF result files and for adjusting global display settings, such as the background color or the default material color.

After importing an STL file, the user can refine the mesh and optionally save the refined version. Once this initial step is complete, the user moves on to the FEM setup. Here, material parameters, loads and boundary conditions are defined. When the setup is finished, the surface mesh is automatically forwarded to the meshing module, which generates a volume mesh suitable for finite element analysis.

The FEM solver then assembles and solves the equations according to the specified inputs. After the computation finishes, the program outputs an XDMF file containing the displacement, stress and strain fields, along with JSON reports that summarize mesh quality and basic simulation metadata. The XDMF file is then loaded back into the application for visualization, allowing users to inspect different result fields directly in the interface.

The application's architecture is organized around a shared state object. This object stores all configuration settings and user selections. It also acts as the communication link between the GUI and the computational components, which include VTK for rendering, Gmsh for meshing and FEniCS for the FEM solver. This structure keeps the system modular and ensures the GUI and backend remain synchronized.

GUI Interaction and Load Assignment

The graphical user interface plays a central role in how users interact with the application. It consists of a rendering window, a menu bar, a headbar and a collapsible sidebar with three tabs: mesh editing, FEM setup and result visualization.

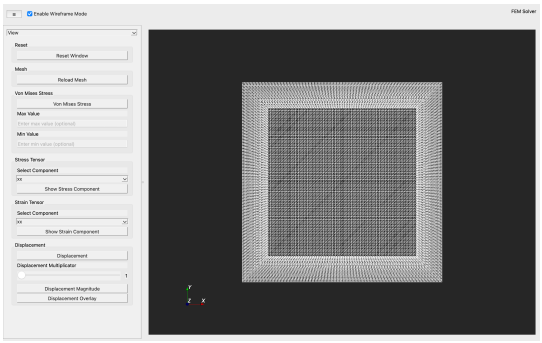

Fig. 1: Overview of the graphical user interface, including the rendering window and visualization tab

The menu bar handles basic file operations and global settings, while the headbar offers quick display controls, such as switching between solid and wireframe views or hiding the sidebar. The sidebar contains most of the tools for refining the mesh, assigning loads, boundary conditions and exploring simulation results.

The Mesh Editing tab allows users to refine the imported geometry. The initial design aimed to include extended functionality for mesh optimization and multi-object manipulation features for scenarios involving moving or interacting bodies. These features were postponed to prioritize the core FEM workflow, but the current structure allows them to be added later.

The FEM Setup tab is where material properties and boundary conditions are defined. Material parameters also determine whether the imported STL file is interpreted in centimetres or metres. Boundary conditions are assigned by choosing an axis and clicking on a vertex in the renderer. All nodes above or below that point along the chosen axis are automatically constrained and the selected region is highlighted in blue.

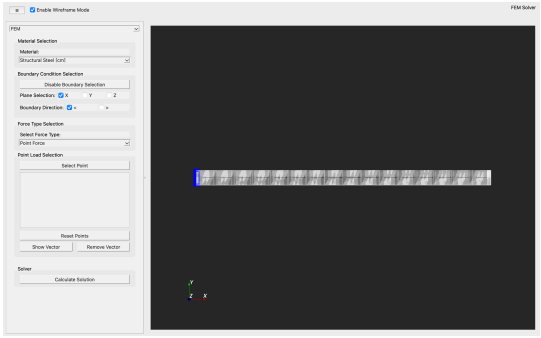

Fig. 2: Example of boundary-condition assignment with the constrained region highlighted in blue

Load assignment works in a similar way. Point loads are added by selecting vertices and entering the corresponding load vectors in the sidebar. Each point load appears in a list, where it can be edited or removed. When a load is selected from the list, it is highlighted in yellow and the others are shown in red. Load directions can also be shown as arrows.

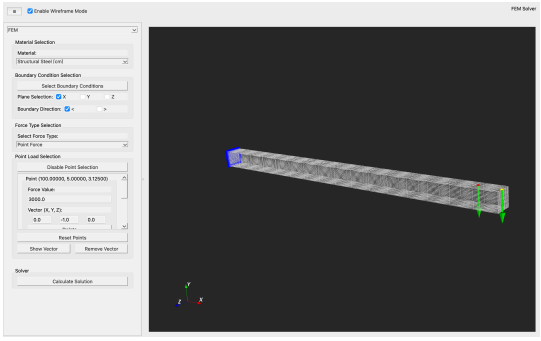

Fig. 3: Point-load assignment interface showing two applied loads with directional arrow indicators

Area loads are assigned by selecting surface triangles individually or using a paint-style selection mode. The same interaction can also be used to remove previously assigned triangles from the load region. A single load vector is then applied to all selected triangles.

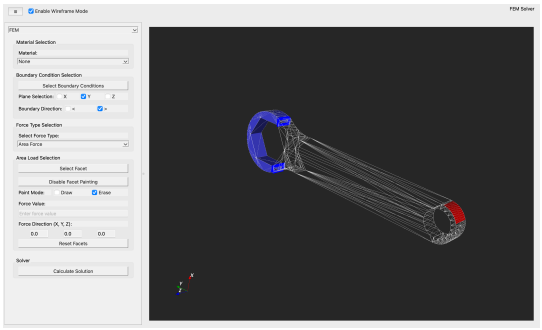

Fig. 4: Area-load assignment using the triangle paint-selection tool to define the loaded region

The Result Visualization tab contains the tools for exploring the simulation output. Users can toggle between the undeformed and deformed mesh, apply displacement exaggeration and view different scalar fields such as von Mises stress, individual stress components, strain components or displacement magnitude. The undeformed and deformed shapes can be displayed together to make the structural response easier to understand. Tooltips and a scalar bar provide numerical values at any point on the mesh, which helps with detailed inspection.

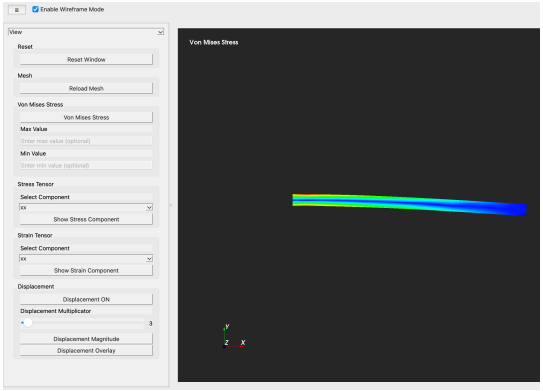

Fig. 5: Visualization of the deformed configuration with von Mises stress distribution and exaggerated deformation

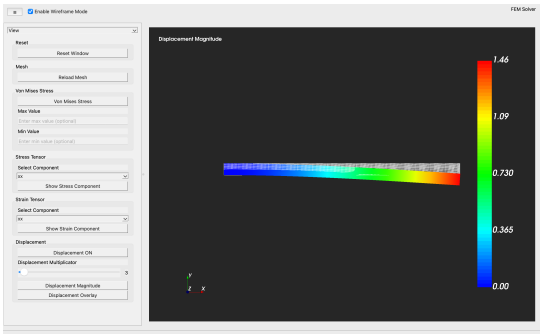

Fig. 6: Visualization of the displacement magnitude field with an exaggerated deformation for improved clarity

Mesh Editing and Geometry Processing

The mesh editing tools prepare STL surface meshes for finite element analysis. Since STL files only store triangular surface faces and do not contain any volume information, they must be converted into a tetrahedral mesh before a simulation can run. In the mesh editing tab, the user can increase the resolution of the surface mesh. The refinement is done through a simple VTK subdivision filter, which produces a denser and smoother surface. More advanced refinement strategies were considered early on, but development time shifted toward other parts of the project.

After the simulation setup is finished, the processed surface mesh is handed over to the meshing backend. This step uses the Gmsh library to create a three-dimensional tetrahedral mesh. During development, this part turned out to be more difficult than expected. Gmsh requires the input geometry to be watertight, manifold and free of issues like holes, gaps, self-intersections, duplicate faces and incorrect normals. Many STL files downloaded from online sources had one or more of these problems, which often caused meshing failures or unstable meshes.

Because of this, we tested several alternative meshing libraries to see if they handled poor-quality STL files better. This investigation took quite a bit of time, but none of the alternatives provided the kind of automatic repair tools that were needed. In the end, Gmsh was still chosen because of its reliable mesh generation and its integration with FEniCS. However, problematic STL inputs remain a challenge. These difficulties point to the need for better automatic diagnostics and repair tools in future versions of the application.

Solver Execution and Backend

When a simulation is started, the surface mesh and all assigned loads and boundary conditions are passed to the backend. The meshing library then creates the tetrahedral volume mesh, which is necessary because finite element calculations require a valid 3D discretization of the geometry.

After successful meshing, the volume mesh and all user-specified parameters are passed to the finite element solver. The implementation is based on linear elasticity, meaning that stresses and strains follow Hooke's law and the structure is modeled through a global linear system. The solver constructs the stiffness matrix for each tetrahedral element and assembles them into the global stiffness matrix. After solving the system, the displacement field becomes available, from which the stress and strain fields are computed. All results are written to XDMF files, along with two small JSON files that store mesh quality data and a summary of the solver settings.

An adaptive numerical strategy is used for solving the linear system. For small and medium-sized meshes, the solver uses a direct method because it is reliable and accurate. For larger meshes, an iterative solver is selected automatically, as iterative methods handle large systems more efficiently and use less memory. This choice helps to avoid picking inappropriate solver options and keeps the computation efficient.

Finite Element Formulation

The finite element formulation is based on standard linear elasticity for three-dimensional solids. In this setting, the material response is described by the relationship between stress and strain, which for isotropic materials follows Hooke's law. When the nodes of the mesh move, each tetrahedral element experiences a strain and this strain is computed using the linear strain-displacement relation. The corresponding stress is then obtained by multiplying the strain with the constitutive matrix, which depends on the material parameters selected by the user.

For every tetrahedral element, a local stiffness matrix is created. This matrix relates nodal displacements to internal forces within that element. All local stiffness matrices are then assembled into one global stiffness matrix representing the entire model. The loads defined by the user form a global force vector. Together, these give the linear system whose unknowns are the nodal displacements. After solving this system, the displacement field is known and all strain and stress components can be computed directly from it. These quantities are used for the visualization in the post-processing stage and are exported to the output directory so they can be inspected or processed further.

Libraries and Technologies Used

The system relies on several software libraries that make the different parts of the workflow possible. The application is written in Python, which simplifies development and provides access to scientific computing tools. The graphical user interface is built with PySide6, which provides the structure for windows, menus and interactive panels. For 3D rendering, the application uses VTK. This library is used to load the STL models, handle user interaction such as selecting vertices or faces and display loads, boundary conditions and simulation results. It also updates the visualization in real time, which is important for creating a smooth user experience.

The tetrahedral meshes used for the FEM solver are generated by GMSH. This library was chosen because, after extensive testing, it turned out to be the most reliable option and offered basic tools for automatically repairing typical STL problems. Meshio is used afterwards to convert the resulting mesh into the file formats required by FEniCS. This ensures smooth data transfer between the meshing stage and the solver. The finite element computations themselves are carried out with FEniCS. This framework provides the mathematical tools for defining the linear elasticity problem and solving it efficiently and it integrates well with Python.

RESULTS

This section evaluates the performance of the application. The experiments were designed to address three primary questions. First, how the computational cost of mesh generation scales with increasing geometric resolution. Second, how the finite element solver behaves in terms of runtime for both point and area loads across different mesh sizes. Third, whether the implementation converges in a consistent manner and reproduces a known analytical reference solution. All runtime measurements were obtained under controlled conditions. For each geometry and refinement level, every operation was executed three times and the reported values correspond to the averaged results. The beam and cube geometries were selected because they allow systematic refinement and represent distinct structural behaviors. The beam exhibits a bending-dominated response under load, whereas the cube shows a more compact, predominantly compressive or shear-dominated behavior. In addition, the beam geometry admits a classical analytical solution for tip deflection, which makes it suitable for validating the numerical results.

Geometry	Level	Points	Elements	Avg time (s)
Beam	coarse	9	24	0.010
Beam	medium	115	558	0.065
Beam	fine	1764	9133	0.815
Beam	super fine	28555	146765	17.318
Beam	extreme fine	511530	2652764	662.142
Cube	coarse	9	24	0.004
Cube	medium	131	609	0.039
Cube	fine	2583	13515	0.740
Cube	super fine	92892	527353	41.210
Dental	-	4156	20695	1.076
Wrench	-	325	1345	0.091

TABLE I: Mesh generation runtime statistics for a range of refinement levels

The first group of experiments examines the runtime of the mesh generation stage. Table 1 lists the number of surface points, the number of tetrahedral elements and the average meshing time for several refinement levels. For the beam, the coarse mesh with 24 elements is produced in approximately 0.01 s, while the super fine mesh with 146,765 elements requires roughly 17.3s. The extreme fine beam mesh contains 2,652,764 tetrahedra, increasing the meshing time to 662.1s. The cube exhibits a comparable trend: the coarse mesh is created in 0.004 s, whereas the super fine cube mesh with 527,353 elements takes 41.2s. These results demonstrate a steep increase in runtime once the element count reaches several hundred thousand. This reflects the additional overhead required to preserve mesh quality and connectivity. The dental and wrench examples further indicate that the system can handle more complex geometries, with meshing times between the medium and fine levels.

Geometry	Level	DOFs	Solver	Avg runtime (s)
Beam	coarse	105	LU	0.052
Beam	medium	2073	LU	0.104
Beam	fine	33372	CG	19.500
Beam	super fine	537894	CG	436.739
Beam	extreme fine	9847821	CG	-
Cube	coarse	105	LU	0.068
Cube	medium	2322	LU	0.151
Cube	fine	51432	CG	5.252
Cube	super fine	2065680	CG	3826.126

TABLE II: Runtime of the finite element solver for point-load simulations on beam and cube meshes

The second set of experiments examines the runtime of the finite element solver. Table 2 presents the average solving times for point-load simulations on the beam and cube using

the same sequence of mesh refinements. For the beam, the coarse system with 105 degrees of freedom is solved using a direct LU-based method in approximately 0.052 s, while the medium system with 2,073 degrees of freedom is solved in 0.104s. Once the mesh reaches the fine level with 33,372 degrees of freedom, the solver switches to an iterative Conjugate Gradient method, increasing the runtime to 19.5s. The super-fine beam mesh with 537,894 degrees of freedom requires 436.7s to converge. At the extreme-fine level, with 9,847,821 degrees of freedom, no converged solution could be obtained within a reasonable time on the available hardware, indicating that this mesh exceeds the practical limits of the current implementation on a standard workstation. The cube data in Table 2 show a similar trend. The fine cube mesh with 51,432 degrees of freedom is solved in 5.25 s, whereas the super-fine mesh with 2,065,680 degrees of freedom needs 3,826.1s. These results suggest that the adaptive choice between direct and iterative solvers is appropriate for the tested problem sizes, while also highlighting the increasing computational cost of large meshes.

Geometry	Level	DOFs	Solver	Avg runtime (s)
Beam	coarse	105	LU	0.031
Beam	medium	2073	LU	0.354
Beam	fine	33372	CG	79.216
Cube	coarse	105	LU	0.044
Cube	medium	2322	LU	0.439
Cube	fine	51432	CG	105.328

TABLE III: Runtime of the finite element solver for area-load simulations on beam and cube meshes.

Table 3 summarises the runtimes for area-load simulations on the same sequence of meshes. The overall trend mirrors the point-load case, although runtimes are consistently higher. For example, the fine beam mesh requires 79.2s for an area load compared to 19.5s for a point load and the cube shows an even larger increase. This behavior likely arises because area loads affect a larger number of nodes, producing a more complex right-hand side in the linear system. However, it could also be related to a less efficient way of handling area loads compared to point loads. These results indicate that both mesh size and load type significantly influence computational performance, with area loads placing higher demands on the solver.

To evaluate the numerical behavior under mesh refinement, convergence tests were performed and are presented in Tables 4 and 5. Two error metrics were used. The energy error, which measures the deviation in total elastic strain energy relative to a reference solution and the displacement error, which captures deviations in nodal displacements. In both cases, the finest mesh serves as the reference solution and is assigned an error of zero. The drop in error at the finest mesh reflects the substantially higher number of degrees of freedom at that refinement level compared with the preceding mesh.

Geometry	Level	DOFs	Energy error E_{err}	Disp. error U_{err}
Beam	coarse	105	0.301	0.082
Beam	medium	2073	0.041	0.032
Beam	fine	33372	0.007	0.015
Beam	super fine	537894	0.000	0.000
Cube	coarse	105	1.000	1.000
Cube	medium	2322	0.933	0.946
Cube	fine	51432	0.747	0.759
Cube	super fine	2065680	0.000	0.000

TABLE IV: Convergence of FEM energy and displacement errors for beam and cube geometries under a point load, using the finest mesh as the reference solution

For point loads, Table 4 shows that the beam begins with an energy error of 0.301 and a displacement error of 0.082 on the coarse mesh. These errors decrease to 0.041 and 0.032 on the medium mesh and further to 0.007 and 0.015 on the fine mesh. The cube follows a similar trend, with errors decreasing steadily as the mesh is refined. This behavior indicates consistent convergence for the tested cases.

Geometry	Level	DOFs	Energy error E_{err}	Disp. error U_{err}
Beam	coarse	105	0.470	0.063
Beam	medium	2073	0.055	0.021
Beam	fine	33372	0.000	0.000
Cube	coarse	105	0.016	0.055
Cube	medium	2322	0.002	0.0003
Cube	fine	51432	0.000	0.000

TABLE V: Convergence of FEM energy and displacement errors for beam and cube geometries under an area load, with errors evaluated relative to the finest mesh

The area-load case in Table 5 shows a similar trend. For the beam, the energy error decreases from 0.470 to 0.055 and the cube already exhibits very small errors on the coarse meshes, approaching zero by the medium level. The consistent decline of both energy and displacement errors supports the conclusion that the implementation converges as expected.

The final part of the evaluation validates the implementation against an analytical solution for a cantilever beam subjected to a point load. The beam is 100 cm long with a square cross-section of 5 cm × 5 cm and is fixed at one end. A downward vertical point load of 2000 N is applied at the free end.

According to Euler-Bernoulli beam theory, the vertical tip deflection of a cantilever beam under an end-point load is

$$\delta = \frac{FL^3}{3EI},$$

where the second moment of area for a rectangular cross-section is

$$I = \frac{bh^3}{12}.$$

With $L = 1\,\mathrm{m}$, $b = h = 0.05\,\mathrm{m}$, $F = 2000\,\mathrm{N}$ and $E = 210 \times 10^9\,\mathrm{Pa}$, the moment of inertia becomes

$$I = 5.21 \times 10^{-7}\,\mathrm{m}^4,$$

yielding a theoretical tip deflection of

$$\delta = 0.006095 \text{ m} = 0.6095 \text{ cm}.$$

This value is used as the reference d_{true} in Table VI.

Geometry	Level	DOFs	d_{FEM}	d_{true}	err_d (%)
Beam	medium	2073	0.5893	0.6095	3.23
Beam	fine	33372	0.6062	0.6095	0.46
Beam	super fine	537894	0.6090	0.6095	0.08

TABLE VI: Validation of the FEM solver for a cantilever beam under a point load, comparing computed displacements with analytical reference values

The medium mesh predicts a displacement of 0.5893 cm, corresponding to an error of 3.23 %. The fine mesh reduces this error to 0.46 % and the super-fine mesh further improves the accuracy to 0.08 %. This progression shows that the solver converges toward the expected physical response as the mesh is refined. The accuracy achieved on the finest mesh suggests a correct implementation of the geometry handling, boundary-condition assignment, load application and linear elasticity formulation in this validation case.

Overall, the results indicate that the system performs reliably across the tested scenarios. Runtime increases significantly with mesh size, particularly for large meshes, while convergence behavior remains consistent under mesh refinement. The analytical validation demonstrates good agreement with the theoretical solution. The main limitations are associated with mesh size and input geometry quality.

Future Work

Although the system provides a complete workflow from geometry import to the visualization of the results, several aspects could be improved based on the limitations observed during development and evaluation.

A primary limitation is the strong dependence on the quality of the input STL files. Many meshes contain defects such as holes, self-intersections, duplicate faces or inconsistent normals, which frequently lead to failures during volume meshing. While the current meshing backend includes basic repair functionality, it is not sufficient for more complex geometries. Future work should therefore focus on the detection of geometric inconsistencies and the integration of more robust repair tools to improve reliability when processing STL data.

Mesh refinement is another area with room for improvement. The current implementation supports only a uniform subdivision strategy, which offers limited control over element distribution. Integrating more advanced refinement approaches would allow users to obtain higher accuracy without unnecessary increases in computational cost. As observed in the results, increasing mesh resolution significantly impacts computational cost. Therefore, adaptive refinement strategies could improve accuracy while limiting the increase in the number of elements.

The physical modelling capabilities of the solver could also be expanded. At present, the system is restricted to linear elasticity with point and area loads. Extending it to handle additional load types and more complex material behavior, such as nonlinear elasticity or thermo-mechanical effects, would make the application applicable to a broader range of engineering problems.

Scalability and performance are further considerations. The results show that very large meshes cause long meshing times or slow visualizations. Introducing parallel computation for mesh generation and solver execution or improving rendering efficiency would help the system handle larger geometries more reliably.

Finally, the graphical interface could be improved to support more precise interaction. Selecting individual points or small surface regions can still be difficult, particularly for complex geometries. Improved selection tools and clearer visual feedback would make the system more intuitive and easier to use.

Overall, these improvements represent the next steps that would increase the flexibility and practical applicability of the application.

Conclusion

This project presented an interactive finite element analysis application that operates directly on STL geometries and guides the user through the workflow from geometry import to the evaluation of displacement, strain and stress fields. By building on widely used open-source libraries such as VTK, Gmsh and FEniCS, the system shows that STL models, which are typically associated with rapid prototyping, can be combined effectively with numerical methods for structural simulation in an accessible and practical way.

The evaluation indicates that the system performs reliably across different geometries and refinement levels. The meshing experiments showed a significant increase in runtime with growing element counts, while solver performance depended strongly on mesh size and load type. The convergence study demonstrated consistent error reduction under mesh refinement and the analytical validation showed good agreement with the theoretical solution.

At the same time, several limitations were identified. The quality of the input STL files has a strong influence on the success of the meshing stage and the current repair tools are limited when handling complex geometries. In addition, surface refinement is restricted to uniform subdivision, which limits control over local mesh density. The solver is also focused on linear elasticity with a small set of load types and very large meshes lead to long computation times and occasionally exceed the capabilities of standard hardware.

Overall, the results show that finite element simulations can be performed directly on STL-based geometries within an interactive workflow. While improvements in geometry handling and computational performance are still required, the developed system provides a solid foundation for further extensions and more advanced applications.

References

[1] F. Béchet, J.-C. Cuilliere, and F. Trochu, "Generation of a finite element mesh from stereolithography (STL) files," *Computer-Aided Design*, vol. 34, no. 1, pp. 1–17, 2002.

[2] M. Müller, M. Teschner, and M. Gross, “Physically-based simulation of objects represented by surface meshes,” in *Proc. Computer Graphics International (CGI)*, 2004, pp. 26–33.
[3] O. C. Zienkiewicz and R. L. Taylor, *The Finite Element Method: Its Basis and Fundamentals*, 6th ed., Oxford, U.K.: Butterworth-Heinemann, 2005.
[4] M. S. Alnæs, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. E. Rognes and G. N. Wells, “The FEniCS Project Version 1.5,” *Archive of Numerical Software*, vol. 3, no. 100, pp. 9–23, 2015.
[5] FEATool Multiphysics, “FEATool Multiphysics User Guide,” [Online]. Available: https://www.featool.com. Accessed: November 2025.
[6] M. Malinen and P. Råback, “Elmer finite element solver for multiphysics and multiscale problems,” in *Multiscale Modelling Methods for Applications in Material Science*, pp. 101–113, 2013.